Sheet6 - 1

```
' Module: DropdownModule
Public Sub CreateDropdowns()
    Dim ws As Worksheet
    Dim lastRow As Long, i As Long, j As Long
    Dim validationList As String
    Dim valCell As Range

    ' Set your target sheet
    Set ws = ThisWorkbook.Sheets("Export through PS")

    validationList = "Bool,Byte,Word,DWord,SInt,Int,DInt,Real"
    lastRow = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row

    Application.EnableEvents = False ' Prevent recursion

    ' --- Section 1: Parameter Name ---
    For i = 1 To lastRow
        If Trim(UCase(ws.Cells(i, 1).Value)) = "PARAMETER NAME" Then
            j = i + 1
            Do While j <= lastRow And Trim(ws.Cells(j, 1).Value) <> "" And _
                     Trim(UCase(ws.Cells(j, 1).Value)) <> "CONSTANT NAME"

                Set valCell = ws.Cells(j, 2)

                ' Add dropdown
                With valCell.Validation
                    .Delete
                    .Add Type:=xlValidateList, AlertStyle:=xlValidAlertStop, _
                         Operator:=xlBetween, Formula1:=validationList
                    .IgnoreBlank = True
                    .InCellDropdown = True
                    .ShowInput = True
                    .ShowError = True
                End With

                j = j + 1
            Loop
        End If
    Next i

    ' --- Section 2: Constant Name ---
    For i = 1 To lastRow
        If Trim(UCase(ws.Cells(i, 1).Value)) = "CONSTANT NAME" Then
            j = i + 1
            Do While j <= lastRow And Trim(ws.Cells(j, 1).Value) <> "" And _
                     Trim(UCase(ws.Cells(j, 1).Value)) <> "CONVEYOR ACTION NAME"

                Set valCell = ws.Cells(j, 2)

                ' Add dropdown
                With valCell.Validation
                    .Delete
                    .Add Type:=xlValidateList, AlertStyle:=xlValidAlertStop, _
                         Operator:=xlBetween, Formula1:=validationList
                    .IgnoreBlank = True
                    .InCellDropdown = True
                    .ShowInput = True
                    .ShowError = True
                End With

                j = j + 1
            Loop
        End If
    Next i

    Application.EnableEvents = True

    MsgBox "Dropdowns created successfully!", vbInformation
End Sub
```

frmDeviceSelect - 1

```
' Assuming your ListBox is named lstDevices
' and a CommandButton named btnOK

Private selectedDevice As String

' Populate the list from array
Public Sub PopulateList(arr As Variant)
    Dim i As Long
    Me.lstDevices.Clear
    For i = LBound(arr) To UBound(arr)
        Me.lstDevices.AddItem arr(i)
    Next i
End Sub

' When user clicks OK
Private Sub btnOK_Click()
    If Me.lstDevices.ListIndex >= 0 Then
        selectedDevice = Me.lstDevices.Value
        Me.Hide
    Else
        MsgBox "Please select a device!"
    End If
End Sub

' Provide function to get selected device
Public Function GetSelectedDevice() As String
    GetSelectedDevice = selectedDevice
End Function
```

Module1 - 1

```
Option Explicit

' === Entry point to run validation ===
Sub HighlightInvalidTokens()
    Dim ws As Worksheet
    Dim lastRow As Long, i As Long
    Dim searchCols As Variant
    Dim deviceName As String

    ' Adjust your sheet name if needed
    Set ws = ThisWorkbook.Sheets("To PS Resource Type")
    lastRow = ws.Cells(ws.Rows.Count, "C").End(xlUp).Row

    ' Columns where tokens may exist
    searchCols = Array(4, 9, 23, 26) ' D=4, I=9, W=23, Z=26

    ' Loop through rows and validate H + AC expressions
    For i = 5 To lastRow
        deviceName = Trim(ws.Cells(i, "C").Value)

        If ws.Cells(i, "H").Value <> "" Then
            Call ColorExpressionTokens(ws.Cells(i, "H"), deviceName, ws, searchCols, lastRow)
        End If

        If ws.Cells(i, "AC").Value <> "" Then
            Call ColorExpressionTokens(ws.Cells(i, "AC"), deviceName, ws, searchCols, lastRow)
        End If
    Next i
End Sub

' === Expression token validator ===
Private Sub ColorExpressionTokens(exprCell As Range, deviceName As String, ws As Worksheet, searchCols
 As Variant, lastRow As Long)
    Dim expr As String, tokens As Variant, token As String
    Dim startPos As Long, t As Long
    Dim matches As Object, m As Object

    expr = exprCell.Value
    exprCell.Font.Color = vbBlack ' reset font

    ' --- Handle RE(...) and FE(...) tokens ---
    With CreateObject("VBScript.RegExp")
        .Global = True
        .IgnoreCase = True
        .Pattern = "\b(RE|FE)\(([A-Za-z0-9_]+)\)"
        If .Test(expr) Then
            Set matches = .Execute(expr)
            For Each m In matches
                token = m.SubMatches(1) ' token inside parentheses
                If Not TokenExists(token, deviceName, ws, searchCols, lastRow) Then
                    ' Highlight the inner token in red
                    startPos = InStr(1, exprCell.Value, token, vbTextCompare)
                    If startPos > 0 Then
                        exprCell.Characters(startPos, Len(token)).Font.Color = vbRed
                    End If
                End If
            Next m
        End If
    End With

    ' --- Remove RE(...) / FE(...) to avoid double validation ---
    expr = Replace(expr, "RE(", " ")
    expr = Replace(expr, "FE(", " ")

    ' Replace multi-character operators
    expr = Replace(expr, ">=", " ")
    expr = Replace(expr, "<=", " ")
    expr = Replace(expr, "NOT", " ")
    expr = Replace(expr, "AND", " ")
    expr = Replace(expr, "OR", " ")
    expr = Replace(expr, "XOR", " ")

    ' Replace single-character operators
```

```
    expr = Replace(expr, "(", " ")
    expr = Replace(expr, ")", " ")
    expr = Replace(expr, ">", " ")
    expr = Replace(expr, "<", " ")
    expr = Replace(expr, "=", " ")
    expr = Replace(expr, "+", " ")
    expr = Replace(expr, "-", " ")
    expr = Replace(expr, "*", " ")
    expr = Replace(expr, "/", " ")

    tokens = Split(expr, " ")

    ' --- Validate plain tokens ---
    For t = LBound(tokens) To UBound(tokens)
        token = Trim(tokens(t))
        If token <> "" Then
            ' Skip numbers
            If IsNumeric(token) Then GoTo NextToken

            ' RE/FE alone without () is invalid
            If UCase(token) = "RE" Or UCase(token) = "FE" Then
                startPos = InStr(1, exprCell.Value, token, vbTextCompare)
                If startPos > 0 Then
                    exprCell.Characters(startPos, Len(token)).Font.Color = vbRed
                End If
                GoTo NextToken
            End If

            ' Normal validation against D/I/W/Z
            If Not TokenExists(token, deviceName, ws, searchCols, lastRow) Then
                startPos = InStr(1, exprCell.Value, token, vbTextCompare)
                If startPos > 0 Then
                    exprCell.Characters(startPos, Len(token)).Font.Color = vbRed
                End If
            End If
        End If
NextToken:
    Next t
End Sub

' === Helper to check if token exists in allowed columns for same device ===
Private Function TokenExists(token As String, deviceName As String, ws As Worksheet, searchCols As Var
iant, lastRow As Long) As Boolean
    Dim j As Long, k As Long
    For j = LBound(searchCols) To UBound(searchCols)
        For k = 5 To lastRow
            If UCase(ws.Cells(k, "C").Value) = UCase(deviceName) Then
                If UCase(ws.Cells(k, searchCols(j)).Value) = UCase(token) Then
                    TokenExists = True
                    Exit Function
                End If
            End If
        Next k
    Next j
    TokenExists = False
End Function
```

Module10 - 1

```
' Module: DropdownModule
Public Sub CreateDropdowns()
    Dim ws As Worksheet
    Dim lastRow As Long, i As Long, j As Long
    Dim validationList As String
    Dim valCell As Range

    ' Set your target sheet
    Set ws = ThisWorkbook.Sheets("Export through PS")

    validationList = "Bool,Byte,Word,DWord,SInt,Int,DInt,Real"
    lastRow = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row

    Application.EnableEvents = False ' Prevent recursion

    ' --- Section 1: Parameter Name ---
    For i = 1 To lastRow
        If Trim(UCase(ws.Cells(i, 1).Value)) = "PARAMETER NAME" Then
            j = i + 1
            Do While j <= lastRow And Trim(ws.Cells(j, 1).Value) <> "" And _
                     Trim(UCase(ws.Cells(j, 1).Value)) <> "CONSTANT NAME"

                Set valCell = ws.Cells(j, 2)

                ' Add dropdown
                With valCell.Validation
                    .Delete
                    .Add Type:=xlValidateList, AlertStyle:=xlValidAlertStop, _
                         Operator:=xlBetween, Formula1:=validationList
                    .IgnoreBlank = True
                    .InCellDropdown = True
                    .ShowInput = True
                    .ShowError = True
                End With

                j = j + 1
            Loop
        End If
    Next i

    ' --- Section 2: Constant Name ---
    For i = 1 To lastRow
        If Trim(UCase(ws.Cells(i, 1).Value)) = "CONSTANT NAME" Then
            j = i + 1
            Do While j <= lastRow And Trim(ws.Cells(j, 1).Value) <> "" And _
                     Trim(UCase(ws.Cells(j, 1).Value)) <> "CONVEYOR ACTION NAME"

                Set valCell = ws.Cells(j, 2)

                ' Add dropdown
                With valCell.Validation
                    .Delete
                    .Add Type:=xlValidateList, AlertStyle:=xlValidAlertStop, _
                         Operator:=xlBetween, Formula1:=validationList
                    .IgnoreBlank = True
                    .InCellDropdown = True
                    .ShowInput = True
                    .ShowError = True
                End With

                j = j + 1
            Loop
        End If
    Next i

    Application.EnableEvents = True

    MsgBox "Dropdowns created successfully!", vbInformation
End Sub
```

Module11 - 1

```
Option Explicit

' === Entry point to run validation and export entire sheet ===
Sub GenerateEntireLB()
    On Error GoTo ErrHandler

    Dim ws As Worksheet
    Dim exprRanges As Variant, refRanges As Variant
    Dim r As Variant
    Dim fDialog As FileDialog, savePath As String
    Dim newWb As Workbook, newWs As Worksheet
    Dim sheetName As String
    Dim invalidFound As Boolean

    ' --- Set sheet ---
    Set ws = ThisWorkbook.Sheets("Export through PS")

    ' --- Ranges to check for expressions ---
    exprRanges = Array("C46:C62", "C79:C92", _
                       "D94:D107", "D109:D126", "D128:D142", _
                       "D144:D157", "D159:D172", "D174:D188", _
                       "D190:D205", "D207:D219", "I24:I44")

    ' --- Reference ranges containing allowed tokens ---
    refRanges = Array("A2:A22", "A46:A62", "A64:A77", "A239:A255", "A221:A237")

    ' --- Validate tokens ---
    invalidFound = False
    For Each r In exprRanges
        If ValidateTokensInRange(ws.Range(r), ws, refRanges) Then
            invalidFound = True
        End If
    Next r

    ' --- Stop if any invalid tokens found ---
    If invalidFound Then
        MsgBox "Validation failed: Invalid token(s) detected. Excel export canceled.", vbCritical
        Exit Sub
    End If

    ' --- Ask user for folder to save Import LB.xlsx ---
    Set fDialog = Application.FileDialog(msoFileDialogFolderPicker)
    With fDialog
        .Title = "Select folder to create 'Import LB.xlsx'"
        If .Show <> -1 Then Exit Sub
        savePath = .SelectedItems(1) & "\Import LB.xlsx"
    End With

    Application.ScreenUpdating = False
    Application.EnableEvents = False
    Application.DisplayAlerts = False

    ' --- Create or open Import LB.xlsx ---
    If Dir(savePath) <> "" Then
        Set newWb = Workbooks.Open(savePath)
    Else
        Set newWb = Workbooks.Add
        newWb.SaveAs Filename:=savePath, FileFormat:=xlOpenXMLWorkbook
    End If

    ' --- Sheet name from Q17 ---
    sheetName = Trim(ws.Range("Q17").Value)
    If sheetName = "" Then sheetName = "Sheet1"

    ' --- Delete existing sheet if it exists ---
    On Error Resume Next
    Set newWs = newWb.Sheets(sheetName)
    If Not newWs Is Nothing Then newWs.Delete
    On Error GoTo 0

    ' --- Add new sheet ---
    Set newWs = newWb.Sheets.Add(After:=newWb.Sheets(newWb.Sheets.Count))
    newWs.Name = sheetName
```

```
    ' --- Copy A1:K263 with all formatting ---
    ws.Range("A1:K263").Copy
    newWs.Range("A1").PasteSpecial Paste:=xlPasteAll
    Application.CutCopyMode = False

    ' --- Save and close workbook ---
    newWb.Save
    newWb.Close

    MsgBox "'Export through PS' copied to '" & savePath & "' in sheet '" & sheetName & "'.", vbInforma
tion

ExitHandler:
    Application.DisplayAlerts = True
    Application.EnableEvents = True
    Application.ScreenUpdating = True
    Exit Sub

ErrHandler:
    MsgBox "Error encountered: " & Err.Description & vbCrLf & _
           "The 'Import LB.xlsx' file was NOT created.", vbCritical
    Resume ExitHandler
End Sub

' === Core validation logic for a range ===
' Returns True if any invalid token is found
Private Function ValidateTokensInRange(exprRange As Range, ws As Worksheet, refRanges As Variant) As B
oolean
    Dim cell As Range
    Dim expr As String, tokens As Variant, token As String
    Dim startPos As Long, t As Long
    Dim matches As Object, m As Object
    Dim invalidDetected As Boolean

    invalidDetected = False

    For Each cell In exprRange
        If Trim(cell.Value) <> "" Then
            expr = cell.Value
            cell.Font.Color = vbBlack ' reset font

            ' --- Handle RE(...) and FE(...) tokens ---
            With CreateObject("VBScript.RegExp")
                .Global = True
                .IgnoreCase = True
                .Pattern = "\b(RE|FE)\(([A-Za-z0-9_]+)\)"

                If .Test(expr) Then
                    Set matches = .Execute(expr)
                    For Each m In matches
                        token = m.SubMatches(1)
                        If Not TokenExistsVertical(token, ws, refRanges) Then
                            startPos = InStr(1, expr, token, vbTextCompare)
                            If startPos > 0 Then
                                cell.Characters(startPos, Len(token)).Font.Color = vbRed
                            End If
                            invalidDetected = True
                        End If
                    Next m
                End If
            End With

            ' --- Remove RE/FE to avoid double validation ---
            expr = Replace(expr, "RE(", " ")
            expr = Replace(expr, "FE(", " ")

            ' --- Replace operators ---
            expr = Replace(expr, ">=", " ")
            expr = Replace(expr, "<=", " ")
            expr = Replace(expr, "NOT", " ")
            expr = Replace(expr, "AND", " ")
            expr = Replace(expr, "OR", " ")
```

```
            expr = Replace(expr, "XOR", " ")
            expr = Replace(expr, "(", " ")
            expr = Replace(expr, ")", " ")
            expr = Replace(expr, ">", " ")
            expr = Replace(expr, "<", " ")
            expr = Replace(expr, "=", " ")
            expr = Replace(expr, "+", " ")
            expr = Replace(expr, "-", " ")
            expr = Replace(expr, "*", " ")
            expr = Replace(expr, "/", " ")

            tokens = Split(expr, " ")

            ' --- Validate plain tokens ---
            For t = LBound(tokens) To UBound(tokens)
                token = Trim(tokens(t))
                If token <> "" Then
                    If IsNumeric(token) Then GoTo NextToken
                    If UCase(token) = "RE" Or UCase(token) = "FE" Then
                        startPos = InStr(1, cell.Value, token, vbTextCompare)
                        If startPos > 0 Then cell.Characters(startPos, Len(token)).Font.Color = vbRed
                        invalidDetected = True
                        GoTo NextToken
                    End If

                    If Not TokenExistsVertical(token, ws, refRanges) Then
                        startPos = InStr(1, cell.Value, token, vbTextCompare)
                        If startPos > 0 Then cell.Characters(startPos, Len(token)).Font.Color = vbRed
                        invalidDetected = True
                    End If
                End If
NextToken:
            Next t
        End If
    Next cell

    ValidateTokensInRange = invalidDetected
End Function

' === Check token against allowed references ===
Private Function TokenExistsVertical(token As String, ws As Worksheet, refRanges As Variant) As Boolea
n
    Dim r As Variant, c As Range
    For Each r In refRanges
        For Each c In ws.Range(r)
            If UCase(c.Value) = UCase(token) Then
                TokenExistsVertical = True
                Exit Function
            End If
        Next c
    Next r
    TokenExistsVertical = False
End Function
```

Module12 - 1

```
Sub FillExternalConnection()
    Dim ws As Worksheet
    Dim userInput As String
    Dim i As Long

    ' Set sheet (adjust if needed)
    Set ws = ThisWorkbook.Sheets("Export through PS")

    ' Ask user for External Connection
    userInput = InputBox("Enter External Connection:", "External Connection")
    If Trim(userInput) = "" Then
        MsgBox "No input provided. Operation cancelled.", vbExclamation
        Exit Sub
    End If

    ' Fill rows 2-22
    For i = 2 To 22
        If ws.Cells(i, "A").Value <> "" Then
            ws.Cells(i, "E").Value = userInput
        End If
    Next i

    ' Fill rows 24-44
    For i = 24 To 44
        If ws.Cells(i, "A").Value <> "" Then
            ws.Cells(i, "E").Value = userInput
        End If
    Next i

    MsgBox "External Connection '" & userInput & "' filled in column E (rows 2-22 and 24-44).", vbInfo
rmation
End Sub
```

Module2 - 1

```
Sub CopyToPLC()
    Dim wsSrc As Worksheet
    Dim wsDest As Worksheet
    Dim lastRow As Long
    Dim i As Long

    ' Set worksheets
    Set wsSrc = ThisWorkbook.Sheets("From Control Design")
    Set wsDest = ThisWorkbook.Sheets("To PLC IO tag")

    ' Find last row in source sheet (based on column C)
    lastRow = wsSrc.Cells(wsSrc.Rows.Count, "C").End(xlUp).Row

    ' Loop through rows from 3 downwards
    For i = 3 To lastRow
        ' Copy C -> B
        wsDest.Cells(i, "B").Value = wsSrc.Cells(i, "C").Value
        ' Copy D -> D
        wsDest.Cells(i, "D").Value = wsSrc.Cells(i, "D").Value
        ' Copy E -> E
        wsDest.Cells(i, "E").Value = wsSrc.Cells(i, "E").Value

        ' Check Name & Address (Name = Column B, Address = Column D)
        If wsDest.Cells(i, "B").Value <> "" And wsDest.Cells(i, "D").Value <> "" Then
            wsDest.Cells(i, "G").Value = True
            wsDest.Cells(i, "H").Value = True
            wsDest.Cells(i, "I").Value = True
        Else
            wsDest.Cells(i, "G").Value = False
            wsDest.Cells(i, "H").Value = False
            wsDest.Cells(i, "I").Value = False
        End If
    Next i
End Sub
```

Module3 - 1

```
Sub CopyControlDesignToResource()
    Dim wsSrc As Worksheet, wsDest As Worksheet
    Dim lastRow As Long, i As Long
    Dim addr As String, firstPart As String, secondPart As String, numPart As String
    Dim rowI As Long, rowQ As Long
    Dim fullName As String
    Dim parts() As String
    Dim j As Long

    ' Set worksheets
    Set wsSrc = ThisWorkbook.Sheets("From Control Design")
    Set wsDest = ThisWorkbook.Sheets("To PS Resource")

    ' Find last row in Control Design based on column E
    lastRow = wsSrc.Cells(wsSrc.Rows.Count, "E").End(xlUp).Row

    ' Initialize starting rows in Resource
    rowI = 5   ' For %I list
    rowQ = 5   ' For %Q list

    ' Optional: clear old data in Resource B:F and I:K from row 5 down
    wsDest.Range("B5:F" & wsDest.Rows.Count).ClearContents
    wsDest.Range("I5:K" & wsDest.Rows.Count).ClearContents

    ' Loop through rows starting at row 3
    For i = 3 To lastRow
        addr = Trim(wsSrc.Cells(i, "E").Value)
        fullName = wsSrc.Cells(i, "C").Value   ' Copy C as-is, do not Trim

        If addr <> "" And fullName <> "" Then
            ' --- Split name into first and rest ---
            parts = Split(fullName, "_")

            firstPart = parts(0) ' before first underscore
            If UBound(parts) >= 1 Then
                ' Rebuild everything after the first underscore
                secondPart = parts(1)
                For j = 2 To UBound(parts)
                    secondPart = secondPart & "_" & parts(j)
                Next j
            Else
                secondPart = "" ' no underscore found
            End If

            ' --- Extract numeric part of address ---
            numPart = Replace(addr, "%I", "")
            numPart = Replace(numPart, "%Q", "")

            ' --- %I case ---
            If Left(addr, 2) = "%I" Then
                wsDest.Cells(rowI, "B").Value = firstPart   ' First part (e.g. Gripper3)
                wsDest.Cells(rowI, "C").Value = secondPart  ' Column C as-is
                wsDest.Cells(rowI, "D").Value = wsSrc.Cells(i, "D").Value
                wsDest.Cells(rowI, "E").Value = numPart
                rowI = rowI + 1
            End If

            ' --- %Q case ---
            If Left(addr, 2) = "%Q" Then
                wsDest.Cells(rowQ, "B").Value = firstPart   ' First part (e.g. Gripper3)
                wsDest.Cells(rowQ, "H").Value = secondPart
                wsDest.Cells(rowQ, "I").Value = wsSrc.Cells(i, "D").Value
                wsDest.Cells(rowQ, "J").Value = numPart
                rowQ = rowQ + 1
            End If
        End If
    Next i
End Sub
```

Module4 - 1

```
Sub FillInputInFK()
    Dim ws As Worksheet
    Dim userInput As String
    Dim i As Long

    Set ws = ThisWorkbook.Sheets("To PS Resource")

    ' Ask user for input
    userInput = InputBox("Enter value to fill in columns F and K:", "Add Value")

    ' Exit if nothing entered
    If userInput = "" Then Exit Sub

    ' Start from row 5 downward
    i = 5
    Do While ws.Cells(i, "B").Value <> ""
        ws.Cells(i, "F").Value = userInput
        ws.Cells(i, "K").Value = userInput
        i = i + 1
    Loop
End Sub
```

Module5 - 1

```
Sub ExportPLCTags_FromControlDesign()
    Dim wsSrc As Worksheet
    Dim wsFrom As Worksheet
    Dim wbDest As Workbook
    Dim wsDest As Worksheet
    Dim wsTagProps As Worksheet
    Dim lastRow As Long
    Dim pasteRow As Long
    Dim userSheetName As String
    Dim i As Long

    ' Ask user for Sheet Name
    userSheetName = InputBox("Enter Sheet Name for Path column:", "Sheet Name")
    If Trim(userSheetName) = "" Then
        MsgBox "No Sheet Name entered. Export cancelled.", vbExclamation
        Exit Sub
    End If

    ' Check if "From Control Design" exists
    On Error Resume Next
    Set wsFrom = ThisWorkbook.Sheets("From Control Design")
    On Error GoTo 0
    If wsFrom Is Nothing Then
        MsgBox "'From Control Design' sheet not found. Export cancelled.", vbCritical
        Exit Sub
    End If

    ' Create new workbook
    Set wbDest = Workbooks.Add
    Set wsDest = wbDest.Sheets(1)
    wsDest.Name = "PLC Tags"

    ' Add headers in PLC Tags
    With wsDest
        .Cells(1, 1).Value = "Name"
        .Cells(1, 2).Value = "Path"
        .Cells(1, 3).Value = "Data Type"
        .Cells(1, 4).Value = "Logical Address"
        .Cells(1, 5).Value = "Comment"
        .Cells(1, 6).Value = "Hmi Visible"
        .Cells(1, 7).Value = "Hmi Accessible"
        .Cells(1, 8).Value = "Hmi Writeable"
        .Cells(1, 9).Value = "Typeobject ID"
        .Cells(1, 10).Value = "Version ID"
    End With

    ' Find last row in From Control Design Column C
    lastRow = wsFrom.Cells(wsFrom.Rows.Count, "C").End(xlUp).Row
    pasteRow = 2

    ' Copy Name (Column C) to PLC Tags Column A
    For i = 3 To lastRow
        If wsFrom.Cells(i, "C").Value <> "" Then
            wsDest.Cells(pasteRow, "A").Value = wsFrom.Cells(i, "C").Value
            wsDest.Cells(pasteRow, "B").Value = userSheetName               ' Path
            wsDest.Cells(pasteRow, "F").Value = True                         ' Hmi Visible
            wsDest.Cells(pasteRow, "G").Value = True                         ' Hmi Accessible
            wsDest.Cells(pasteRow, "H").Value = True                         ' Hmi Writeable
            pasteRow = pasteRow + 1
        End If
    Next i

    ' Copy Data type (Column D) to PLC Tags Column C
    For i = 3 To lastRow
        If wsFrom.Cells(i, "D").Value <> "" Then
            wsDest.Cells(i - 1, "C").Value = wsFrom.Cells(i, "D").Value
        End If
    Next i

    ' Copy Logical address (Column E) to PLC Tags Column D with % prefix
    For i = 3 To lastRow
        If wsFrom.Cells(i, "E").Value <> "" Then
            wsDest.Cells(i - 1, "D").Value = "%" & wsFrom.Cells(i, "E").Value
```

```
        End If
    Next i

    ' Create TagTable Properties sheet
    Set wsTagProps = wbDest.Sheets.Add(After:=wbDest.Sheets(wbDest.Sheets.Count))
    wsTagProps.Name = "TagTable Properties"

    ' Add headers
    wsTagProps.Cells(1, 1).Value = "Path"
    wsTagProps.Cells(1, 2).Value = "BelongsToUnit"
    wsTagProps.Cells(1, 3).Value = "Accessibility"

    ' Add user Sheet Name in Path row 2
    wsTagProps.Cells(2, 1).Value = userSheetName

    ' Auto-fit columns
    wsDest.Columns("A:J").AutoFit
    wsTagProps.Columns("A:C").AutoFit

    ' Ask user where to save
    Dim savePath As Variant
    savePath = Application.GetSaveAsFilename( _
        InitialFileName:="PLCTags.xlsx", _
        FileFilter:="Excel Files (*.xlsx), *.xlsx")
    If savePath = False Then
        MsgBox "Export cancelled.", vbInformation
        wbDest.Close False
        Exit Sub
    End If

    ' Save and close
    wbDest.SaveAs Filename:=savePath, FileFormat:=xlOpenXMLWorkbook
    wbDest.Close

    MsgBox "PLC Tags exported successfully to: " & savePath, vbInformation
End Sub
```

Module6 - 1

```
Sub CopyPSResourceData()
    Dim wsSource As Worksheet, wsTarget As Worksheet
    Dim startIndex As Long, endIndex As Long
    Dim srcRow As Long, tgtRow As Long
    ' Set sheets
    Set wsSource = ThisWorkbook.Sheets("To PS Resource")
    Set wsTarget = ThisWorkbook.Sheets("To PS Resource Type")
    ' Ask for start and end index
    startIndex = CLng(InputBox("Enter Start Row Index from 'To PS Resource':"))
    endIndex = CLng(InputBox("Enter End Row Index from 'To PS Resource':"))
    If startIndex <= 0 Or endIndex < startIndex Then
        MsgBox "Invalid range entered!", vbExclamation
        Exit Sub
    End If
    ' Start target pasting from row 5
    tgtRow = 5
    ' Loop through the given range
    For srcRow = startIndex To endIndex
        wsTarget.Cells(tgtRow, "D").Value = wsSource.Cells(srcRow, "C").Value
        wsTarget.Cells(tgtRow, "E").Value = wsSource.Cells(srcRow, "D").Value
        wsTarget.Cells(tgtRow, "F").Value = wsSource.Cells(srcRow, "H").Value
        wsTarget.Cells(tgtRow, "G").Value = wsSource.Cells(srcRow, "I").Value
        tgtRow = tgtRow + 1
    Next srcRow
    MsgBox "Data copied successfully!", vbInformation
End Sub
```

Module7 - 1

```
Sub SelectDeviceAndCopyData()
    Dim wsSrc As Worksheet, wsDest As Worksheet, wsDevice As Worksheet
    Dim lastRowSrc As Long, lastRowDevice As Long
    Dim i As Long, lastCol As Long, insertRow As Long
    Dim arr As Variant, dict As Object
    Dim entryValue As String, splitValue As String, devicePart As String
    Dim selectedDevice As String
    Dim foundCell As Range

    ' --- Step 1: Prepare device list ---
    Set wsDevice = ThisWorkbook.Sheets("From PS Kinematics Data")
    lastRowDevice = wsDevice.Cells(wsDevice.Rows.Count, "A").End(xlUp).Row

    ' Collect unique devices
    Set dict = CreateObject("Scripting.Dictionary")
    For i = 2 To lastRowDevice
        If wsDevice.Cells(i, 1).Value <> "" Then
            If Not dict.exists(wsDevice.Cells(i, 1).Value) Then
                dict.Add wsDevice.Cells(i, 1).Value, 1
            End If
        End If
    Next i
    arr = dict.keys

    ' --- Step 2: Show UserForm for selection ---
    frmDeviceSelect.PopulateList arr
    frmDeviceSelect.Show
    selectedDevice = frmDeviceSelect.GetSelectedDevice

    If selectedDevice = "" Then
        MsgBox "No device selected. Operation cancelled."
        Exit Sub
    End If

    ' --- Step 3: Copy & insert data for selected device ---
    Set wsSrc = ThisWorkbook.Sheets("From Control Design")
    Set wsDest = ThisWorkbook.Sheets("Export through PS")

    lastRowSrc = wsSrc.Cells(wsSrc.Rows.Count, "E").End(xlUp).Row
    lastCol = wsDest.Cells(1, wsDest.Columns.Count).End(xlToLeft).Column

    Set foundCell = wsDest.Columns(1).Find(What:="Entry Name", LookIn:=xlValues, LookAt:=xlWhole)
    If foundCell Is Nothing Then
        MsgBox "'Entry Name' not found in destination sheet."
        Exit Sub
    End If
    insertRow = foundCell.Row + 1

    For i = 3 To lastRowSrc
        If Left(wsSrc.Cells(i, "E").Value, 1) = "I" Then
            entryValue = wsSrc.Cells(i, "C").Value

            ' Split at first underscore
            If InStr(entryValue, "_") > 0 Then
                devicePart = Left(entryValue, InStr(entryValue, "_") - 1)
                splitValue = Mid(entryValue, InStr(entryValue, "_") + 1)
            Else
                devicePart = entryValue
                splitValue = entryValue
            End If

            ' Only process rows matching selected device
            If StrComp(devicePart, selectedDevice, vbTextCompare) = 0 Then
                wsDest.Rows(insertRow).Insert Shift:=xlDown
                wsDest.Cells(insertRow, 1).Value = splitValue
                wsDest.Cells(insertRow, 2).Value = wsSrc.Cells(i, "D").Value
                wsDest.Cells(insertRow, 3).Value = wsSrc.Cells(i, "C").Value

                With wsDest.Range(wsDest.Cells(insertRow, 1), wsDest.Cells(insertRow, lastCol))
                    .Interior.Color = RGB(255, 255, 255)
                    .Font.Bold = False
                End With
```

```
                insertRow = insertRow + 1
            End If
        End If
    Next i

    MsgBox "Data transfer complete for device: " & selectedDevice
End Sub
```

Module8 - 1

```
Sub SelectDevice()
    Dim ws As Worksheet
    Dim lastRow As Long, i As Long
    Dim dict As Object
    Dim arr As Variant
    Dim selectedDevice As String

    ' Source sheet
    Set ws = ThisWorkbook.Sheets("From PS Kinematics Data")

    ' Last row in column A
    lastRow = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row

    ' Collect unique devices
    Set dict = CreateObject("Scripting.Dictionary")
    For i = 2 To lastRow
        If ws.Cells(i, 1).Value <> "" Then
            If Not dict.exists(ws.Cells(i, 1).Value) Then
                dict.Add ws.Cells(i, 1).Value, 1
            End If
        End If
    Next i
    arr = dict.keys

    ' Populate UserForm and show
    frmDeviceSelect.PopulateList arr
    frmDeviceSelect.Show

    ' Get selected device
    selectedDevice = frmDeviceSelect.GetSelectedDevice
    If selectedDevice <> "" Then
        MsgBox "You selected: " & selectedDevice
    Else
        MsgBox "No device selected."
    End If
End Sub
```

Module9 - 1

```
Option Explicit

Sub SelectDeviceAndCopyDataUpd()
    Dim wsSrc As Worksheet, wsDest As Worksheet, wsDevice As Worksheet
    Dim lastRowSrc As Long, lastRowDevice As Long
    Dim i As Long, insertRow As Long, insertRowExit As Long
    Dim entryValue As String, splitValue As String, devicePart As String
    Dim selectedDevice As String
    Dim foundEntry As Range, foundExit As Range
    Dim drivingJointList As String, poseList As String
    Dim skipRows As Variant, r As Long
    Dim rng As Range

    ' --- Step 1: Setup sheets ---
    Set wsDevice = ThisWorkbook.Sheets("From PS Kinematics Data")
    Set wsDest = ThisWorkbook.Sheets("Export through PS")

    lastRowDevice = wsDevice.Cells(wsDevice.Rows.Count, "A").End(xlUp).Row

    ' --- Step 2: Create dropdown in Q11 from device list ---
    With wsDest.Range("Q11")
        On Error Resume Next
        .Validation.Delete
        On Error GoTo 0
    End With

    Set rng = wsDevice.Range("A2:A" & lastRowDevice)
    wsDest.Range("Q11").Validation.Add Type:=xlValidateList, _
        AlertStyle:=xlValidAlertStop, Operator:=xlBetween, Formula1:="=" & rng.Address(True, True, , T
rue)

    ' --- Step 3: Get selected device from Q11 ---
    selectedDevice = Trim(wsDest.Range("Q11").Value)
    If selectedDevice = "" Then
        MsgBox "No device selected in Q11. Please select a device from dropdown.", vbExclamation
        Exit Sub
    End If

    ' --- Step 4: Clear previous data (rows 2:261, A:K) except skipRows ---
    skipRows = Array(1, 23, 45, 63, 78, 93, 108, 127, 143, 158, 173, 189, 206, 220, 238, 256, 258, 260
)

    For r = 2 To 261
        If IsError(Application.Match(r, skipRows, 0)) Then
            wsDest.Range("A" & r & ":K" & r).ClearContents
        End If
    Next r

    ' --- Step 5: Copy Inputs/Outputs from Control Design ---
    Set wsSrc = ThisWorkbook.Sheets("From Control Design")
    lastRowSrc = wsSrc.Cells(wsSrc.Rows.Count, "E").End(xlUp).Row

    ' Find Entry and Exit anchors
    Set foundEntry = wsDest.Columns(1).Find(What:="Entry Name", LookIn:=xlValues, LookAt:=xlWhole)
    Set foundExit = wsDest.Columns(1).Find(What:="Exit Name", LookIn:=xlValues, LookAt:=xlWhole)

    If foundEntry Is Nothing Or foundExit Is Nothing Then
        MsgBox "'Entry Name' or 'Exit Name' not found in destination sheet.", vbCritical
        Exit Sub
    End If

    insertRow = foundEntry.Row + 1
    insertRowExit = foundExit.Row + 1

    ' --- Inputs (I) ---
    For i = 3 To lastRowSrc
        If Left(wsSrc.Cells(i, "E").Value, 1) = "I" Then
            entryValue = wsSrc.Cells(i, "C").Value
            'entryValueAddress = wsSrc.Cells(i, "E").Value
            If InStr(entryValue, "_") > 0 Then
                devicePart = Left(entryValue, InStr(entryValue, "_") - 1)
                splitValue = Mid(entryValue, InStr(entryValue, "_") + 1)
            Else
```

```
            devicePart = entryValue
            splitValue = entryValue
        End If
        If StrComp(devicePart, selectedDevice, vbTextCompare) = 0 Then
            wsDest.Cells(insertRow, 1).Value = splitValue
            wsDest.Cells(insertRow, 2).Value = wsSrc.Cells(i, "D").Value
            wsDest.Cells(insertRow, 3).Value = wsSrc.Cells(i, "C").Value
            wsDest.Cells(insertRow, 7).Value = wsSrc.Cells(i, "E").Value
            insertRow = insertRow + 1
        End If
    End If
Next i

' --- Outputs (Q) ---
For i = 3 To lastRowSrc
    If Left(wsSrc.Cells(i, "E").Value, 1) = "Q" Then
        entryValue = wsSrc.Cells(i, "C").Value
        If InStr(entryValue, "_") > 0 Then
            devicePart = Left(entryValue, InStr(entryValue, "_") - 1)
            splitValue = Mid(entryValue, InStr(entryValue, "_") + 1)
        Else
            devicePart = entryValue
            splitValue = entryValue
        End If
        If StrComp(devicePart, selectedDevice, vbTextCompare) = 0 Then
            wsDest.Cells(insertRowExit, 1).Value = splitValue
            wsDest.Cells(insertRowExit, 2).Value = wsSrc.Cells(i, "D").Value
            wsDest.Cells(insertRowExit, 3).Value = wsSrc.Cells(i, "C").Value
            wsDest.Cells(insertRowExit, 7).Value = wsSrc.Cells(i, "E").Value
            insertRowExit = insertRowExit + 1
        End If
    End If
Next i

' --- DrivingJoint & Pose collection ---
drivingJointList = ""
poseList = ""
For i = 2 To lastRowDevice
    If wsDevice.Cells(i, 1).Value = selectedDevice Then
        Select Case LCase(wsDevice.Cells(i, 2).Value)
            Case "drivingjoint"
                If drivingJointList = "" Then
                    drivingJointList = wsDevice.Cells(i, 3).Value
                Else
                    drivingJointList = drivingJointList & "," & wsDevice.Cells(i, 3).Value
                End If
            Case "pose"
                If poseList = "" Then
                    poseList = wsDevice.Cells(i, 3).Value
                Else
                    poseList = poseList & "," & wsDevice.Cells(i, 3).Value
                End If
        End Select
    End If
Next i

' --- DrivingJoint dropdowns ---
If drivingJointList <> "" Then
    Dim rngDrive As Variant, r1 As Variant
    rngDrive = Array("C109:C126", "C128:C142", "C144:C157", _
                     "C159:C172", "C207:C219", "C221:C237", "C239:C255")
    For Each r1 In rngDrive
        With wsDest.Range(r1)
            On Error Resume Next
            .Validation.Delete
            On Error GoTo 0
            .Validation.Add Type:=xlValidateList, AlertStyle:=xlValidAlertStop, _
                            Operator:=xlBetween, Formula1:=drivingJointList
        End With
    Next r1
End If

' --- Pose dropdowns ---
```

```
    If poseList <> "" Then
        Dim rngPose As Variant, r2 As Variant
        rngPose = Array("C94:C107", "D188:D204", "D221:D237")
        For Each r2 In rngPose
            With wsDest.Range(r2)
                On Error Resume Next
                .Validation.Delete
                On Error GoTo 0
                .Validation.Add Type:=xlValidateList, AlertStyle:=xlValidAlertStop, _
                                Operator:=xlBetween, Formula1:=poseList
            End With
        Next r2
    End If

    ' --- Display selected device in Q17 ---
    With wsDest.Range("Q17")
        .Value = selectedDevice
        .Font.Bold = True
        .Font.Size = 18
    End With

    MsgBox "Data transfer complete in Export through PS for device: " & selectedDevice, vbInformation
End Sub
```